



MP305 Network Flow Models II

```
In [1]: from IPython.display import display, Math, Latex
```

Overview

This file contains a number of Python functions for finding the maximal flow through a network G subject to minimal cost using the Ford Fulkerson Algorithm.

The network graph G is stored in a set G of two element tuples (i,j) describing the directed arcs (i,j) of G .

It is assumed that node number 1 is the source and the greatest node N_{sink} is the sink.

Thus $G=\{(1,2), (2,3), (1,3)\}$ describes a network with 3 nodes where node 1 is the source and node 3 is the sink.

The capacity $c(i,j)$, flow $\phi(i,j)$ and cost $l(i,j)$ of the arc (i,j) of G are stored in $c[i][j]$, $\phi[i][j]$ and $l[i][j]$. Here c , ϕ and l are Python lists.

Python Functions

The `Initialise(G)` function

Having defined the network G , initialise c , ϕ and l values to zero via the `Initialise` function before defining their values in any particular example. The global variable N_{sink} , the sink node of G , is also found by the `Initialise` function.

The main `Iterate(G)` function

This implements the full algorithm to find the maximum flow with minimal cost.

```
In [ ]:
```

The `Iterate(G)` function is based on a number of other Python functions:

`Flows(G)`

This checks for conservation of flow and prints out all of the current flows for `G` and the total cost of this flow.

`Links(G)`

This finds all arcs `(i, j1)`, `(i, j2)`, ... *out* of node `i` of `G`. The nodes `j1, j2, ..` are stored in a global list of sets `Out`.

`SourceSink(G)`

This finds all of the paths from source to sink in any network `G` and results in a global set `SinkPaths` of such paths.

`IncremNet(G)`

This finds the Incremental Network `Gp` associated with the current flow of the network `G`.

`Newflows(G)`

This updates the flows `phi` of `G` according to the best chain found through `Gp`. If the maximal flow is reached, then this is indicated and the maximum flow value is outputted. Otherwise, the output is: the change in flow (`eps`), the cost of the best chain, and the best chain.

`Iterate(G)`

Implements the full algorithm to find the maximum flow with minimal cost. The output is as follows:

- (1) The incremental network `Gp`.
- (2) The paths through `Gp` from source to sink.
- (3) The output of `Newflows(G)`.

(4) The output of `Flows(G)` giving the current flows and cost of `G`.

In []:

```
In [2]: def Initialise (Gin):
        global c,phi,l,cp,lp,Nsink
        Nsink=1
        for arc in Gin:
            i,j=arc
            Nsink=max(Nsink,i,j)
        # for convenience c[i][j] is capacity of arc [i,j]
        c=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        phi=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        l=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        cp=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        lp=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        print("All values of c,phi and l initialised to zero")
```

```
In [3]: def Flows (Gin):
        global Nsink,l,phi
        Flowin=[0 for i in range(Nsink+1)]
        Flowout=[0 for i in range(Nsink+1)]
        for arc in Gin:
            i,j=arc
            Flowin[j] = Flowin[j] + phi[i][j]
            Flowout[i] = Flowout[i] + phi[i][j]
        for k in range(2,Nsink):
            if Flowin[k] != Flowout[k]:
                print("*** ERROR *** Flow not conserved at node", k)
        if Flowout[1] != Flowin[Nsink]:
            print("*** ERROR *** Flow not conserved at source or sink")
        Totalcost = 0
        for arc in Gin:
            i,j=arc
            phi_ij = phi[i][j]
            Totalcost = Totalcost + l[i][j]*phi_ij
            print(arc," has flow ",phi_ij)
        print("Total Cost is ", Totalcost)
```

```
In [4]: def Links (Gin):
        global Nsink,Out
        Out=[set() for k in range(Nsink)] # labelled 0..Nsink-1
        for arc in Gin:
            i,j = arc
            Out[i - 1] = Out[i - 1] | set([j])
```

```
In [5]: def SourceSink(Gin):
        # finds all paths SinkPaths from source 1 to sink Nsink of network G
        global Nsink,SinkPaths
        Links(Gin)
        Paths = set() # current paths from source stored as set of tuples
        SinkPaths = set() # paths from source to sink Nsink stored as set of tuple
```

```

path = 1 # source node label
for node in Out[0]:# need out edge from node 1
    pathn = (path,node)
    if node == Nsink:
        SinkPaths = SinkPaths | set([pathn])
    else:
        Paths = Paths | set([pathn])
Npaths = len(Paths)
while (0 < Npaths):
    NewPaths = set()
    for oldpath in Paths:
        nold = len(oldpath)
        m = oldpath[-1] # last node in tuple oldpath
        for mout in Out[m-1]:
            if not mout in oldpath:
                if mout == Nsink:
                    SinkPaths = SinkPaths | set([oldpath+tuple([Nsink])])
                else:
                    NewPaths = NewPaths | set([oldpath+tuple([mout])])
    Paths = NewPaths
    Npaths = len(Paths)
print("Paths from source to sink: ",SinkPaths)

```

```

In [6]: def Newflows(Gin):
# A procedure to modify original flows on Gin along SinkPaths of Gp with minin
global Gp,phi,c,l,cp,lp,ArcSign,Out
SourceSink(Gp)
if SinkPaths == set():
    Links(Gin)
    Flow = 0
    for node in Out[0]:
        Flow = Flow + phi[1][node]
    Cost=0
    for arc in Gin:
        i,j=arc
        Cost=Cost+l[i][j]*phi[i][j]
    print("Maximal flow found:", Flow, " with minimal cost ", Cost)
else:
    for k in range(len(SinkPaths)):
        cost = 0
        epset = set()
        path=list(SinkPaths)[k]
        for n in range(0, len(path)-1):
            i = path[n]; j = path[n+1]; epset = epset | set([cp[i][j]]);
        eps = min(tuple(epset))
        if k == 0: # first path
            mincost = cost; bestpath = path; besteps = eps
        elif cost < mincost:
            mincost = cost; bestpath = path; besteps = eps
    print("A best path in Gp is ", bestpath, " of minimum cost ", mincost)
    print("The min capacity on this path is epsilon ", besteps)
    print("The min cost is ", mincost)
    for k in range(0, len(bestpath) - 1):

```

```

        i = bestpath[k]; j = bestpath[k+1]
        if ArcSign[i][j] == 1:
            phinewij = phi[i][j] + beststeps; phi[i][j]=phinewij
        else:
            phinewji=phi[j][i] = phi[j][i] - beststeps; phi[j][i]=phinewji
    #print("Flow=",Flow)

```

```

In [7]: def Iterate(Gin):
        IncremNet(Gin)
        Newflows(Gin)
        for arc in Gin:
            i,j=arc
            print((i,j)," flow = ", phi[i][j])

```

```

In [8]: def IncremNet(Gin):
        # procedure to create incremental network Gp from given flow network G
        global Gp,Nsink,phi,c,l,cp,lp,ArcSign
        # define lists for ArcSign, cp and lp (indexed by 0..Nsink-1)
        cp=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        lp=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        ArcSign=[[0 for i in range(Nsink+1)] for j in range(Nsink+1)]
        Gp=set([])
        for arc in Gin:
            i,j=arc
            pij = phi[i][j]; pji = phi[j][i]; cij = c[i][j]; lij = l[i][j]
            if (pij < cij and (pji == 0 or not (j,i) in Gin)): # ij arc
                #Gp edges, capacitites and costs added
                cpij = cij - pij; cp[i][j] = cpij; lpij = lij; lp[i][j] =lpij
                ArcSign[i][j] = 1
                Gp=Gp | {(i,j)}
            if pij>0: # ji arc
                cpji = pij; cp[j][i] = cpji; lpji=-lij; lp[j][i] = lpji
                ArcSign[j][i] = -1
                Gp=Gp | {(j,i)}
        print("Incremental Network:",Gp)

```

```

In [ ]:

```

Q 1. Power Station/Coal Field Supply

Find the maximal flow for minimal cost for the network below where (capacity,cost) is shown:

Network

This is the Coalfield/Power station supply problem discussed in the notes

* Find the incremental network, its capacities and costs at each iteration.

```
In [9]: G={ (1,2), (1,3), (2,4), (2,5), (3,4), (3,5), (4,6), (5,6) }
```

```
In [10]: Initialise(G)
```

All values of c, phi and l initialised to zero

Input capacities

```
In [11]: c[1][2]=3; c[1][3]=5; c[2][4]=3; c[2][5]=3  
c[3][5]=5; c[3][4]=5; c[4][6]=2; c[5][6]=5
```

Input costs

```
In [12]: l[1][2]=5; l[1][3]=3;  
l[2][4]=3  
l[2][5]=6; l[3][4]=5  
l[3][5]=9
```

```
In [13]: Flows(G)
```

```
(2, 4) has flow 0  
(1, 2) has flow 0  
(3, 4) has flow 0  
(4, 6) has flow 0  
(5, 6) has flow 0  
(2, 5) has flow 0  
(1, 3) has flow 0  
(3, 5) has flow 0  
Total Cost is 0
```

```
In [14]: Iterate(G)
```

```

Incremental Network: {(2, 4), (1, 2), (2, 5), (3, 4), (5, 6), (4, 6), (1, 3),
(3, 5)}
Paths from source to sink: {(1, 2, 5, 6), (1, 3, 4, 6), (1, 2, 4, 6), (1, 3,
5, 6)}
A best path in Gp is (1, 3, 4, 6) of minimum cost 8
The min capacity on this path is epsilon 2
The min cost is 8
(2, 4) flow = 0
(1, 2) flow = 0
(3, 4) flow = 2
(4, 6) flow = 2
(5, 6) flow = 0
(2, 5) flow = 0
(1, 3) flow = 2
(3, 5) flow = 0

```

Continue to iterate until max flow for minimal cost found

```
In [15]: Iterate(G)
```

```

Incremental Network: {(2, 4), (1, 2), (3, 4), (4, 3), (3, 1), (6, 4), (5, 6),
(2, 5), (1, 3), (3, 5)}
Paths from source to sink: {(1, 2, 5, 6), (1, 2, 4, 3, 5, 6), (1, 3, 5, 6)}
A best path in Gp is (1, 2, 5, 6) of minimum cost 11
The min capacity on this path is epsilon 3
The min cost is 11
(2, 4) flow = 0
(1, 2) flow = 3
(3, 4) flow = 2
(4, 6) flow = 2
(5, 6) flow = 3
(2, 5) flow = 3
(1, 3) flow = 2
(3, 5) flow = 0

```

```
In [16]: Iterate(G)
```

```

Incremental Network: {(2, 4), (2, 1), (3, 4), (4, 3), (6, 5), (3, 1), (6, 4),
(5, 6), (1, 3), (3, 5), (5, 2)}
Paths from source to sink: {(1, 3, 5, 6)}
A best path in Gp is (1, 3, 5, 6) of minimum cost 12
The min capacity on this path is epsilon 2
The min cost is 12
(2, 4) flow = 0
(1, 2) flow = 3
(3, 4) flow = 2
(4, 6) flow = 2
(5, 6) flow = 5
(2, 5) flow = 3
(1, 3) flow = 4
(3, 5) flow = 2

```

```
In [17]: Iterate(G)
```

```
Incremental Network: {(2, 4), (2, 1), (3, 4), (4, 3), (6, 5), (3, 1), (6, 4),  
(5, 3), (1, 3), (3, 5), (5, 2)}  
Paths from source to sink: set()  
Maximal flow found: 7 with minimal cost 73  
(2, 4) flow = 0  
(1, 2) flow = 3  
(3, 4) flow = 2  
(4, 6) flow = 2  
(5, 6) flow = 5  
(2, 5) flow = 3  
(1, 3) flow = 4  
(3, 5) flow = 2
```

In []:

Q2. (*)

A road network is shown below with the capacity and time taken per car on each road indicated.

Network

* Find the maximal flow through the network for minimal total travel time for all cars from A to B.

* Compare this to the flow from B to A.

Note: You can use the network and capacities you created in the first lab Network Flows I, Question 3.

In []:

In []:

Q.3 (*) Soft Drinks Stock Control

A soft drinks firm buys fruit at the beginning of each month i at a cost per 100kg p_i in units of 1000 Euro.

The firm can store up 2000kg of fruit at any given time where cost in units 1000 Euro of refrigeration per month per 100kg is r_i .

The consumption requirements are c_i per month in 100kg units.

Based on last year's figures, the following estimates have been made:

	Jan	Feb	Mar	April	May	June	July	Aug	Sept	Oct	Nov	Dec
p_i	18	17	17	15	12	8	7	6	9	12	14	17
r_i	1	1	2	2	3	5	6	6	5	3	2	1
c_i	9	6	6	7	11	14	16	18	15	10	7	6

In []:

(a) Find the best purchasing schedule starting from January based on these estimates assuming that the firm has no fruit in storage on Jan 1st.

How much will fruit cost for the year and how much will be spent on refrigeration?

(b) Suppose that the firm has 500kg of fruit in storage at the beginning of the year.

What is the best purchasing schedule that ensures that 500kg of fruit is again in storage at the very end of the year?

In []:

In []:

In []: